A Compact Python-like Compiler to a Sane AArch64 Subset

Normalization, specialization, and residual code generation

16 August 2026

Abstract

This note presents a compact compiler from a small Python-like language to readable AArch64 assembly. The source language has integer literals, variables, arithmetic, sequential constant bindings, and return. Its normalizer performs constant propagation, beta-like substitution of known bindings, arithmetic folding, and dead binding elimination. Code generation then emits a conservative function using a small register set and a frame pointer. The example demonstrates how a dependently typed normalizer's staging idea can be applied to a conventional compiler: normalize static structure first, then interpret or compile the residual program.

1 Source language

Programs have the form

$$\mathsf{let}\ x = e;\ \ldots\ \mathsf{return}\ e.$$

Expressions are

$$e ::= n \mid x \mid e + e \mid e - e \mid e * e \mid (e).$$

There are no implicit conversions, objects, calls, mutation, exceptions, or hidden runtime allocation. This restriction is intentional: it makes the source-to-target correspondence explicit.

2 Normalization

The normalizer evaluates an expression against a constant environment ρ:

$$\mathsf{norm}_\rho(n) = n, \qquad \mathsf{norm}_\rho(x) = \rho(x) \text{ when defined,}$$

and folds a binary operation whenever both normalized operands are literals. A binding whose normalized right-hand side is a literal is installed in ρ and omitted from the residual program. Thus

$$\mathsf{let}\ x = 2 + 3 * 4;\ \mathsf{let}\ y = x * 10;\ \mathsf{return}\ y + 1$$

normalizes to the single return expression 141.

This is the first-order analogue of normalization by evaluation: static syntax is evaluated into a canonical value and then reified as residual syntax. The same separation appears in the dependently typed core, where quotation is inert until the evaluator crosses an explicit stage boundary.

3 AArch64 target subset

The emitter uses only:

```
mov x0, #constant
mov x1, x0
ldr x0, [x29, #-offset]
str x0, [x29, #-offset]
add/sub/mul x0, x0, x1
stp/ldp x29, x30, [sp, #...]
ret
```

The frame pointer is established at entry. Intermediate values use $x0$ and $x1$; bindings use negative frame-pointer offsets. The target is deliberately conservative and readable rather than optimized for instruction count.

4 Compiled example

The implementation is `pyarm64.cpp`. It is built with:

```
g++ -std=c++23 -Wall -Wextra -pedantic -O2 pyarm64.cpp -o pyarm64
./pyarm64 > sample_arm64.s
```

The normalized result is 141. The emitted assembly is:

```
.text
.global main
main:
  stp x29, x30, [sp, #-16]!
  mov x29, sp
  mov x0, #141
  ldp x29, x30, [sp], #16
  ret
```

The host verification checks the presence of the text section, global entry point, normalized constant, frame restoration, and return instruction. The assembly is structurally verified on the current host; assembling and executing it requires an AArch64 toolchain or target machine.

5 Type and staging interpretation

The source AST can be viewed as an indexed family $\mathsf{Expr}(\tau)$ where the supported result type is an integer type. A richer version would index effects and calling conventions as well:

$$\mathsf{compile} : \mathsf{Expr}(\tau, \epsilon) \to \mathsf{Asm}(\tau, \epsilon).$$

Normalization is type preserving because it substitutes only values already assigned the binding's type. The residual emitter consumes the same typed result. This prevents a future extension from compiling a boolean, aggregate, or effectful expression through the integer-only path accidentally.

6 Limitations and next steps

The prototype does not yet support function definitions, parameters, conditionals, booleans, arrays, strings, system calls, or register allocation. The principled extension path is to add typed expressions and explicit effects first, then lower closures and control flow to a documented ABI subset. Each extension should preserve the invariant that normalization and code generation agree on the indexed result type.